CASE STUDY · SHOWCASE EDITION

Legacy SAS → CDP Migration

Volume 1

Engagement, Discovery & Architecture

Context, scope, discovery method and the architecture before and after migration

WHAT THIS VOLUME COVERS

The client context and scope, the constraints and risks under which the work ran, how an undocumented engine was reverse-engineered, the original architecture, the behavioural differences of the target platform, and the bridge-layer design that absorbed them.

ABOUT THIS DOCUMENT

"Meridian Media" is a hypothetical organisation. All client names, data values, file names and code identifiers have been anonymised or generalised to protect client identity and intellectual property. Figures are illustrative and rounded; the method and sequence of work are described as executed.

Client Analytics · Project Documentation Set (3 volumes)

Contents

1. Executive Summary

Meridian Media, a hypothetical multi-channel publisher, moved its marketing data from a legacy Oracle warehouse to a cloud Customer Data Platform (CDP). Every campaign-scoring model the business relied on was fed by a single SAS feature engine — a macro-driven pipeline that builds one wide table of 475 customer-level variables. The engine had been inherited from a prior vendor with no documentation.

The engagement was scoped as a short remapping exercise: point the existing SAS jobs at the CDP and replace table and field names. It became a discovery and modernisation programme. The first production run on the CDP produced **~5,000 errors** and only **344 of 475 variables populating (72%)**. Seven verified iterations later, the pipeline ran with **zero errors** and **475 of 475 variables populating (100%)**, reconciled against the legacy platform and signed off by the business.

Measure	Start	End
Execution errors	~5,000	0
Variables populating	344 / 475 (72.4%)	475 / 475 (100%)
Original recipe logic rewritten to accommodate the platform	—	None
Files carrying platform-specific changes	—	2 bridge files
Sampled magazine revenue understated	~35%	Restored
Production campaigns disrupted	—	None

1.1 What made the difference

- **Discovery before migration.** The architecture, call graph and variable lineage were reconstructed from the code itself, with AI used to read the codebase and logs at a speed no manual review could match.
- **A bridge layer instead of rewrites.** Two new files absorbed every platform difference; the client's 500+ recipe calls were left as written.
- **Diagnosis at the source.** Every defect was proven by profiling the CDP columns directly before any code changed. Two confident first explanations were overturned this way.
- **Assurance by census, not by log.** A variable-level population census and a normalised legacy comparison replaced the execution log as the definition of 'done'.
- **Governance of meaning.** Fixes that changed what a number means were taken to the business owner with evidence and recorded in a decision register.

1.2 How this documentation is organised

Volume	Title	Contents
1	Engagement, Discovery & Architecture	Context, scope, constraints, discovery method, original and target architecture, bridge-layer design
2	Diagnosis & Remediation	Diagnostic method, fault taxonomy, full defect register, bridge specifications, population progression, change control
3	Assurance, Delivery & Lessons	QC framework, test harness, performance, production safety, governance, timeline, outcomes, 20 lessons, 8 takeaways, reusable playbook

2. Client Context and Engagement Scope

2.1 The client

Meridian Media sells magazine subscriptions, one-shot books, continuity programmes (series shipped on a schedule) and greeting cards, and runs direct-mail fundraising. Its marketing team selects audiences for each campaign and scores them with predictive models. Each model consumes a subset of a shared feature set — recency, frequency, monetary value, product affinity, promotion history, channel and tenure — produced by the SAS engine described in this volume.

2.2 The business driver

The legacy Oracle warehouse was being retired in favour of a Spark-based CDP. Campaign calendars did not pause for the migration, so the engine had to produce equivalent outputs on the CDP while live campaigns continued to be scored. Any material drift in a model input — a revenue variable understated, a tenure date shifted — would silently change who was mailed.

2.3 Scope as contracted versus scope as discovered

Dimension	As scoped	As discovered
Nature of work	Table and field name replacement	Reverse-engineering, remediation and parity proof
Documentation	Assumed adequate	None — logic existed only in code
Platform differences	Naming only	Naming, NULL semantics, date types, temp-table behaviour, reference data
Definition of done	Jobs run without error	100% variable population, reconciled to legacy, business sign-off
Duration	About one month	About four months, in seven iterations

> **KEY INSIGHT**
> The timeline was driven by discovery, not by replacement. Until the system was understood well enough to reason about, no migration estimate could be credible.

2.4 Stakeholders and roles

The delivery team worked in three roles. Names are deliberately omitted in this showcase edition.

Role	Responsibilities
Engagement lead	Client communication, prioritisation, escalation, scope and status reporting
Analytics engineer	Code reading, diagnosis, remediation, bridge-layer design, run execution
QC lead	Population census, attribute comparison, legacy reconciliation, QC workbooks
Client modelling lead	Variable definitions, business meaning, sign-off on meaning-changing fixes
Client platform team	CDP objects, data loads, reference data, data-availability items

3. Constraints, Risks and Working Conditions

3.1 Constraints

- **Shared production platform.** Diagnostics and test runs executed against the same CDP environment that served live campaigns.
- **Undocumented code.** Five interdependent SAS files with no design notes; the only specification was the code and its outputs.
- **Two environments, similar names.** Test and production connections, schemas and folder trees shared naming patterns, raising the risk of running against the wrong one.
- **Long feedback loop.** A full run took about an hour regardless of audience size (see Volume 3).
- **Moving data.** The CDP was still receiving migration loads and daily feeds throughout the engagement.

3.2 Risk register (as maintained)

Risk	Impact	Mitigation
Fix applied to wrong environment	Production output corrupted	Verify connection, schema and path before every run; isolated test outputs
Silent value drift	Models scored on wrong inputs	Population census + value-level legacy reconciliation
Overwriting the last good output	Loss of a known-good baseline	Copy to protected name before every risky run
Recipe edits altering client logic	Client loses ownership; regressions	Bridge layer; recipes changed only on business request
Meaning changes shipped unilaterally	Loss of client trust	Decision register and business sign-off
Reference data drift	Run stops or variables silently empty	Integrity check; neutral holding group; lookup refresh

4. Discovery Method

No design document existed, so the system's design had to be derived from the system itself. Discovery proceeded in four steps, each producing an artefact the next step depended on.

4.1 Step 1 — Parse the include chain

The caller program includes an orchestrator, which in turn includes the library files in a fixed order. Establishing that order revealed ownership: which file defines connections, which defines reference lookups, which generates SQL, and which defines variables. Load order later became a design tool — the bridge files work precisely because they load after the code they adjust.

4.2 Step 2 — Map the macro call graph

The library contains 21 SQL-builder macros and 16 variable-category macros, which together make over 500 recipe calls. Each recipe call was traced to the builder that emits its SQL, producing a map from every output variable to the code path that creates it.

4.3 Step 3 — Capture the emitted SQL

The engine writes each generated query to its log. Reading the emitted SQL, rather than the source macros, showed exactly what reached the database. This distinction proved decisive: several defects were invisible in source and obvious in the emitted predicate.

4.4 Step 4 — Rebuild the lineage

Combining the call graph with emitted SQL produced a lineage for every variable: source table, filter conditions and the argument slot each condition occupies, join keys, date window, and aggregation. This lineage became the reference for every subsequent diagnosis.

4.5 Where AI fitted

AI was used where the bottleneck was comprehension, not typing.

Activity	How AI was used	Human role
Code comprehension	Traced include chain and call graph across 500+ calls in hours	Validated against known outputs
Log forensics	Classified and counted ~5,000 errors; tied cascades to root predicates	Chose which hypothesis to test
Diagnostic scripting	Drafted read-only probes for each hypothesis	Reviewed, ran and interpreted
Documentation	Drafted architecture notes and registers as work happened	Edited and approved
Decisions	None	All mappings, fixes and sign-offs

> **PRINCIPLE**
> AI read, traced, counted and drafted; the team decided, verified and signed off. On two occasions the model's confident first explanation was overturned by profiling the source.

5. Original Architecture

5.1 Layered structure

Layer	Component	Responsibility
Caller	driver.sas	Credentials, build date, audience keycode, output name, list of variables to build
Orchestration	orchestrator.sas	Assigns connections, defines run executors (full pull, audience pull, point-in-time save), loads the library
Library	definitions.sas (~1,100 lines)	Product and campaign lookups, utility macros, formats, reference-code integrity check
Library	sql_engine.sas (~1,950 lines)	21 query builders; execute, stage and merge macros
Library	variables.sas (~1,650 lines)	16 variable-category macros; 500+ recipe calls producing 475 output columns
Source	Legacy Oracle warehouse	Orders, order lines, subscriptions, promotions, audience log, product master

5.2 The per-variable execution loop

For each requested variable the engine performs the same four steps, approximately 2,400 times in a full run:

1. **Build** — a builder assembles SQL from the recipe's filter slots, date window and code lists.
2. **Execute** — the query runs on the database through a pass-through connection.
3. **Stage** — the result lands in a temporary table keyed on customer.
4. **Merge** — the staged result is left-joined onto the wide output table by customer ID; helper columns are dropped at the end.

5.3 Filters live in numbered slots

Recipes do not write SQL. They pass conditions into positional macro arguments. Some slots carry equality conditions, some carry an operator with a value (sign slots), and two carry exclusions. A shared branch in each builder turns each slot into a predicate. This design is efficient — and it means one defective branch affects every recipe that uses that slot. On this engine, the exclusion slots alone served 118 requested variables.

5.4 Dates stored as offsets

Date variables are written as the number of days between the event and the run's build date, not as calendar dates. Two outputs built on different dates therefore hold different numbers for the same event. Any comparison between legacy and CDP outputs had to convert both sides to calendar dates first (Volume 3, section 1).

5.5 Design consequence

> **WHY ONE FIX COULD MOVE DOZENS OF VARIABLES**
> Because SQL is generated at run time by shared builders, a defective pattern in one builder multiplies across every variable it serves — and a single correction there repairs all of them at once. The reverse is also true: a flawed fix at builder level breaks everything it touches.

6. Target Platform and Behavioural Differences

The CDP is a Spark-based platform reached over ODBC, organised into an analytic schema (curated views keeping legacy-style names) and a core schema (platform objects with renamed, camel-case columns).

Area	Legacy Oracle	Cloud CDP	Consequence
Column names	UPPER_SNAKE, as referenced by recipes	Core objects renamed; some re-typed	Recipes emitted names that did not exist — the source of most of the ~5,000 errors
NULLs	Key filter columns effectively always populated	Many columns migrated with NULLs	Bare ≠ filters silently dropped rows (ANSI three-valued logic)
Date types	DATETIME throughout	Mix of DATE and TIMESTAMP	Offset arithmetic produced wrong values without error
Temporary objects	Native temp tables	Server-side staging tables in a shared schema	Leftover tables could block later runs
Reference data	Stable	New codes added; legacy codes retired	Integrity-check stops; silently empty variables
Field semantics	Created date = capture date	Some timestamps record load time, not event time	Migrated records carried the migration date

7. Bridge Architecture

7.1 The design decision

Two approaches were available. The first was to edit the recipes directly so they referenced CDP names and semantics. The second was to leave the recipes untouched and translate at the boundary. The bridge approach was chosen for four reasons:

- **Ownership** — the client's analysts can still read and maintain the recipes they wrote.
- **Reviewability** — every platform accommodation sits in two files, not scattered across thousands of lines.
- **Reversibility** — a wrong mapping is a one-line correction in one place.
- **Risk** — recipes that already worked could not be broken by edits they never received.

7.2 Current architecture: seven files

Order	File	Status	Role
1	driver.sas	Original	Run configuration
2	orchestrator.sas	Original + guards	Connections to two CDP schemas; stale-table guard; include chain
3	definitions.sas	Original + reference codes	Lookups; neutral holding group for unseen codes
4	sql_engine.sas	Original + NULL-safe predicates	Builders emit NULL-safe not-equal in every slot
5	compat_bridge.sas	New	Name mapping, join wrappers, value quoting, reference-key lookup, purpose-built recipes
6	date_bridge.sas	New	Date-type detection and guarded conversion
7	variables.sas	Original + one ported recipe	Recipes; changed only where the business requested a change of meaning

7.3 How the bridges work

Both bridge files load after the SQL engine and before the variable recipes. They override engine macros by redefinition: when the recipes run, they call the bridge versions, which resolve names, quote values and convert dates correctly before handing off to the original logic. The engine file itself is never modified for platform reasons.

> **RULE**
> Translate at the boundary, not inside the logic. Recipes are touched only when the business asks for a change of meaning — never to accommodate the platform.

7.4 Verification of the include chain

Because the bridges work by load order and redefinition, a missing or truncated bridge file fails quietly. After every deployment, a fresh SAS session loads the chain and confirms that every macro the bridges should define exists. This check was introduced after an incident described in Volume 2 (defect D09).

8. Glossary

Term	Meaning in this document
Audience	The set of customers a run builds variables for, identified by a keycode in the audience log
Build date	The reference date of a run; date variables are stored as days before it
Bridge file	A file that loads after the engine and overrides macros to absorb platform differences
Census	Classification of every output column as populating, all-zero or all-blank
CDP	Customer Data Platform — the target cloud platform
Emitted SQL	The SQL text the engine actually generates and logs at run time
Exclusion slot	A recipe argument position carrying a not-equal / not-in condition
Sign slot	A recipe argument position carrying an operator and a value
Harness	A small, isolated test audience that reproduces the full failure profile
Populating	A variable with at least one non-missing, non-zero value across the audience